

Similarity-Aware Defense Scheme for Online Brute-Force Attacks in Encrypted Deduplication

Guanxiong Ha^{1b}, Chunfu Jia^{1b}, Xiaowei Ge^{1b}, Yuchen Chen, and Xuan Shan^{1b}

Abstract—Encrypted deduplication is appealing to cloud storage systems, as it provides both storage savings and data confidentiality. Unfortunately, it is vulnerable to brute-force attacks (BFAs), which poses a severe security threat. Server-aided message-locked encryption (MLE) was proposed to mitigate BFAs by deploying a key server to perform rate limiting for key generation requests. However, the existing rate-limiting strategy has several limitations: (1) Vulnerable to Sybil BFAs; (2) Improper rate-limiting thresholds; (3) Inability to identify requests for duplicate data. To address these limitations, we present a similarity-aware rate-limiting strategy based on the following observations: (i) the candidate data used for online BFAs are generally similar; (ii) there won't be a lot of similar data in a short time period in real-world datasets. Then, we propose two similarity-aware defense schemes, SADS-I and SADS-II. SADS-I incorporates data similarity into the key generation and rate limiting using a partially oblivious pseudorandom function (POPRF) but leaks data similarity. SADS-II leverages Intel SGX to protect data similarity and improve performance. Through evaluation using real-world datasets, we demonstrate that our schemes exhibit high data upload performance and provide stronger security protection against online BFAs compared to the existing server-aided MLE.

Index Terms—Cloud storage, encrypted deduplication, brute-force attacks (BFAs), rate limiting, data similarity.

I. INTRODUCTION

CLOUD storage is an attractive way for individuals or enterprises to cope with surging volumes of data [1], [2], [3], [4], [5]. With the unprecedented growth of data, many cloud storage service providers such as Dropbox [6], Mozy [7], and Google Drive [8] leverage *deduplication* to improve storage efficiency [9]. It works by deduplicating all data and only storing a single physical copy of duplicate data to reclaim storage space. Cloud service providers can leverage deduplication to reduce the storage space of primary storage by more than 50% [10] and backup storage by more than 90% [11]. Nowadays, users

tend to outsource encrypted data to the cloud to protect data privacy. Unfortunately, conventional encryption algorithms convert identical plaintexts into fully random ciphertexts, which impedes deduplication. As a result, given the premise of data privacy protection, achieving efficient deduplication remains a challenging problem.

Message-locked encryption (MLE) [12] is a common approach to achieve encrypted deduplication. It derives encryption keys from data contents to ensure that identical plaintexts can be encrypted into identical ciphertexts. Nevertheless, MLE can only provide *convergent security* [13], meaning that it can only provide confidentiality for *unpredictable data* (i.e., the plaintext space cannot be exhausted) [13]. For *predictable data*, such as employment offer letters [14] or medical test files [15], MLE is vulnerable to the *offline brute-force attack (BFA)*. Suppose that the target file F underlying a ciphertext C_F can be drawn from a dictionary of candidate files $D_F = \{F_1, F_2, \dots, F_n\}$. If an adversary possesses D_F , it could generate all candidate keys based on D_F locally and then restore F from C_F , as the key generation process depends solely on the data content. To address this issue, Bellare et al. [13] propose *server-aided MLE*, which utilizes a dedicated key server to control the rate of key generation. Clients are required to interact with the key server for key generation, thus preventing offline BFAs. In the case of an adversary launching *online BFAs* by sending a large number of key generation requests, the key server can apply *rate limiting* to mitigate the attack.

Although server-aided MLE is considered the state-of-the-art approach to mitigate online BFAs in encrypted deduplication, its rate-limiting strategy still has the following limitations (abbreviated as **L**):

L1. Vulnerable to Sybil BFAs: In *Sybil BFAs*, an adversary creates multiple Sybil accounts [16], [17], [18] or compromises multiple clients as Sybil accounts, using these Sybil accounts to launch a large number of key generation requests to the key server (see Fig. 1). The existing server-aided MLE implements *per-client rate limiting*, wherein the key server applies rate limiting based on the number of requests initiated by each client. Consequently, the existing rate-limiting cannot detect Sybil BFAs as each Sybil account sends only a small number of requests.

L2. Improper rate-limiting thresholds: Server-aided MLE sets a uniform rate-limiting threshold for all data belonging to a client. Nevertheless, a uniform threshold is inappropriate,

Received 25 April 2025; revised 5 August 2025; accepted 9 September 2025. Date of publication 12 September 2025; date of current version 14 January 2026. This work was supported in part by National Key R&D Program of China under Grant 2018YFA0704703 and in part by National Natural Science Foundation of China under Grant 62172238. (Corresponding author: Chunfu Jia.)

Guanxiong Ha, Chunfu Jia, Xiaowei Ge, Yuchen Chen, and Xuan Shan are with the College of Cryptology and Cyber Science, Nankai University, Tianjin 300350, China, and also with the Key Laboratory of Network and Data Security Technology, and Key Laboratory of Data and Intelligent System Security (DISec) Ministry of Education, Tianjin 300350, China (e-mail: cfjia@nankai.edu.cn).

Digital Object Identifier 10.1109/TDSC.2025.3609514

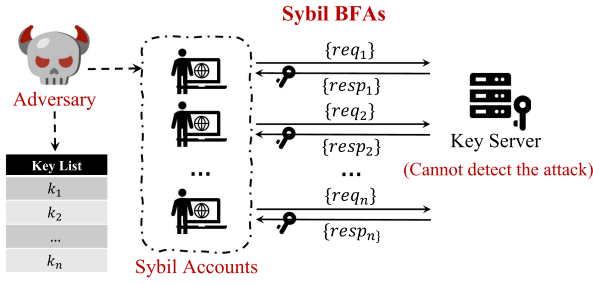


Fig. 1. Sybil BFAs in server-aided MLE. The adversary uses multiple Sybil accounts to send key generation requests to circumvent rate limiting.

since the entropy of different files is different. Suppose two predictable files F_1 and F_2 can be drawn from the dictionaries $D_1 = \{F_{11}, \dots, F_{1n}\}$ and $D_2 = \{F_{21}, \dots, F_{2n'}\}$, respectively. An adversary can obtain all candidate keys for F_1 or F_2 by submitting n or n' key generation requests to the key server. Assuming that $n \gg n'$, then a uniform threshold for both F_1 and F_2 is improper.

L3. Inability to identify requests for duplicate data: In server-aided MLE, the key server cannot identify the key generation requests for duplicate data, as all requests are blinded by random values. Consequently, the key server may count multiple times for many key generation requests corresponding to duplicate data. Requests for duplicate data are legitimate rather than malicious, as they do not grant the client more keys. However, the key server may improperly enforce rate limiting due to the excessive number of requests for duplicate data, resulting in performance degradation.

A. Our Contributions

In this paper, we first present two key observations for online BFAs: (1) the candidate data used for online BFAs are generally similar; (2) there won't be a lot of similar data in a short time period in real-world datasets. These observations motivate us to present *similarity-aware rate-limiting strategy*, which performs rate limiting based on the number of key generation requests corresponding to similar data rather than the number of requests per client. This rate-limiting strategy can alleviate Sybil BFAs and set flexible thresholds to achieve secure and accurate rate limiting. Moreover, we propose two *similarity-aware defense schemes*, namely SADS-I and SADS-II. Both of them achieve similarity-aware rate-limiting. SADS-I incorporates data similarity into key generation and rate limiting based on the *partially oblivious pseudorandom function (POPRF)* [19], [20], [21]. The key is derived from the data similarity, data content, and a secret key managed by the key server. The limitation of SADS-I is that it leaks data similarity to the key server and cannot identify requests for duplicate data. SADS-II leverages the *trusted execution environment (TEE)* provided with Intel SGX to address the limitations of SADS-I. SADS-II can protect data similarity, identify key generation requests for duplicate data, and improve performance by avoiding the use of computationally expensive cryptographic primitives. In summary, we make the following contributions:

- Based on two key observations for online BFAs, we present a similarity-aware rate-limiting strategy, which performs rate limiting based on the number of requests corresponding to similar data.
- We design two similarity-aware defense schemes, SADS-I and SADS-II. The former utilizes POPRF to incorporate data similarity into key generation and rate limiting but leaks data similarity and lacks the ability to identify requests for duplicate data. The latter uses Intel SGX to address the limitations of SADS-I and avoid the use of computationally expensive cryptographic primitives to improve performance.
- We conduct extensive experiments using multiple real-world datasets. The results show that our similarity-aware rate-limiting strategy provides stronger security protection than the existing rate-limiting strategy when an adversary launches online BFAs. Moreover, both SADS-I and SADS-II have high data upload performance.

II. BACKGROUND AND MOTIVATION

In this section, we present the background of encrypted deduplication and two key observations for online BFAs.

A. Encrypted Deduplication

Deduplication improves storage efficiency by reclaiming the storage space for redundant data [9], [22]. This paper focuses on chunk-level deduplication due to its high storage efficiency. It divides files into multiple chunks of several KB in size and identifies each chunk by the cryptographic hash (called the *fingerprint*) of the chunk content. Encrypted deduplication adds an encryption layer on top of deduplication. Bellare et al. [12] propose MLE to achieve encrypted deduplication, which derives encryption keys from data contents. A prominent instantiation of MLE is the convergent encryption (CE) [23]. CE utilizes the data's hash as the key for performing symmetric encryption, achieving efficient encrypted deduplication.

Although MLE efficiently achieves encrypted deduplication, it is subject to the inherent limitation of deterministic encryption algorithms and is vulnerable to offline BFAs [13]. To address this issue, server-aided MLE [13] deploys a key server. During the key generation, a client and the key server respectively input a data hash h and a secret key sk to run *oblivious pseudorandom function (OPRF)* [24]. After OPRF, the client learns a server-aided MLE key k_M derived from $\text{PRF}_{sk}(h)$, where $\text{PRF}_{sk}(\cdot)$ denotes a pseudorandom function (PRF) with a secret key sk . The introduction of the key server prevents offline BFAs, and the key server can rate-limit key generation requests from clients to mitigate online BFAs. Although Liu et al. propose PAKED-edup [25] to resist BFAs without deploying the key server, their scheme requires a large number of clients to be online, which is unrealistic in practice. Currently, most encrypted deduplication schemes [3], [26], [27], [28], [29] defend against BFAs based on server-aided MLE. However, as stated in Section I, the existing rate-limiting strategy in server-aided MLE still has limitations.

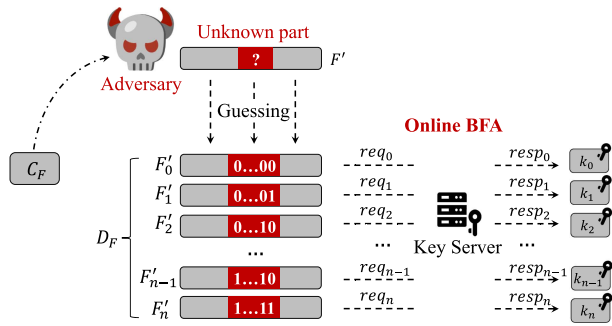


Fig. 2. Online BFAs for the target file F underlying the ciphertext C_F . The adversary learns the encrypted file C_F by intercepting communication or compromising the cloud server and has a file F' similar to F . Then it constructs a file dictionary D_F containing many similar candidate files to launch online BFAs.

B. Our Observations

We present two key observations for online BFAs that motivate us to propose the similarity-aware rate-limiting strategy. The first observation is that *the candidate data used for online BFAs are generally similar*. Specifically, to launch online BFAs on a target file F , the adversary is likely to know most of the contents of it and then constructs the dictionary of candidate files based on the remaining few contents (see Fig. 2). That is, a rational adversary needs to possess a file F' that is similar to the target file F before launching online BFAs. Without a similar file, constructing all possible candidate files will be costly, and online BFAs will be significantly inefficient. Moreover, all candidate files used for online BFAs are similar, with the only difference being the parts that are unknown to the adversary. Consider the following scenario: Alice uploads her encrypted employment offer letter to the cloud, and her colleague Trudy wishes to learn her salary from the letter. If Trudy intercepts the encrypted letter and Alice's salary can be drawn from a finite set (i.e., the salary is predictable [14], [15], [30]), she can create candidate files using all possible salaries of Alice to launch online BFAs. Specifically, Trudy computes hashes of all candidate files and then sends key generation requests to the key server to learn all candidate keys for restoring Alice's letter. In this case, Trudy possesses an employment offer letter that is similar to Alice's letter. Furthermore, all the candidate files created by Trudy are similar to each other in terms of their content, with the only difference being the salary information. Li et al. [30] have verified that similar attack methods are feasible in real-world scenarios. They simulate side-channel attacks in deduplicated cloud storage systems under both LAN and cloud setups. In the example they presented, the adversary can learn the specific data content of its target file by forging 841 similar candidate files.

The above observation motivates us to design a similarity-aware rate-limiting strategy. If a large number of key generation requests for similar data are detected, then the rate limiting is performed to mitigate potential online BFAs. However, since the similarity-aware rate limiting is a global-scale rate limiting mechanism, we need to consider the issue of false positives. In other words, whether the similarity-aware rate limiting might be

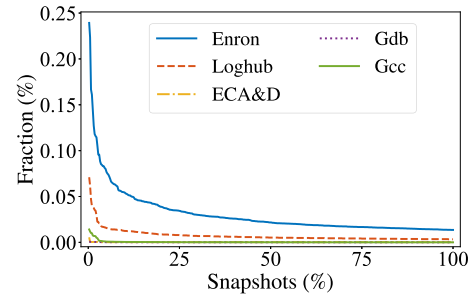


Fig. 3. Proportion of the number of chunks corresponding to each feature over the total number of chunks; the x -axis represents the percentile of snapshots, which are sorted by the fraction in descending order.

enforced to degrade the data upload performance of legitimate users when there is no online BFAs.

To analyze false positives, we present the second observation that *there won't be a lot of similar data in a short time period in real-world datasets*. We analyze five real-world datasets: (1) *Enron* [31], which contains email data from about 150 users. (2) *Loghub* [32], which maintains a collection of system logs. (3) *ECA&D* [33], which contains a series of daily observations at meteorological stations throughout Europe and the Mediterranean. (4) *Gcc* [34], which contains Gnu compiler collection code data. (5) *Gdb* [35], which contains Gnu project debugger code data. We perform content-defined chunking for each file in these datasets using Rabin fingerprinting [36]. The minimum, average, and maximum chunk sizes are set to 4 KB, 8 KB, and 16 KB, respectively. After chunking, all data are divided into variable-size chunks. We then compute a *feature* for each chunk using similarity detection algorithms [37], [38] (described in Section IV-D) to identify similar chunks. Similar chunks have the same feature. We calculate the proportion of the number of chunks corresponding to each feature over the total number of chunks.

As shown in Fig. 3, there are only very few chunks that correspond to the same feature. In Enron, Loghub, Gcc, ECA&D and Gdb, at most 0.24%, 0.07%, 0.014%, 0.005% and 0.001% of chunks correspond to the same feature, respectively. Consequently, the amount of similar data in real-world datasets is very small, and the presence of a large number of requests for similar data is more likely to indicate the occurrence of online BFAs rather than legitimate usage. The small amount of similar data in the real-world datasets is due to the fact that the content of these real-world data (such as emails, logs and climate observations) has strong timeliness or is affected by spatiotemporal dynamics.

The second observation illustrates that *the false positive rate of the similarity-aware rate limiting is likely to be low*. This means that if there is no online BFA, the similarity-aware rate limiting is unlikely to be enforced, and hence, the data upload performance of legitimate users will not degrade. Note that Li et al. [30] recently propose a similar observation that practical workloads are less likely to have many similar data chunks. Their research [30] provides the basis for our observation. The difference between our work and Li et al. [30] is that they focus on the defense of side-channel attacks [14] in client-side deduplication, while we aim to mitigate online BFAs. Additionally, their

analysis primarily involves source code data, while we analyze more types of data, including email, climate, and log data.

III. PRELIMINARIES

In this section, we provide the notations used in this paper and outline the non-interactive zero knowledge proofs (NiZK), POPRF, and Intel SGX, which are the building blocks of our schemes.

A. Notations

Let λ and ε be the security parameter and empty string respectively. We denote by $(u, v) \leftarrow P(A(x), B(y))$ the protocol P engaged by the parties A and B with inputs x and y , and outputs u and v respectively. We use $||$ to denote the concatenation of bit strings. By $y \leftarrow \text{Alg}(x_1, x_2, \dots, x_n)$, we mean to run the algorithm Alg on inputs $x_1, x_2, \dots, x_n$, with the output of y . $\text{Enc}_{\text{SE}}(k, \cdot)$ and $\text{Dec}_{\text{SE}}(k, \cdot)$ denote the symmetric encryption and symmetric decryption algorithms with key k , respectively.

B. Non-Interactive Zero Knowledge Proofs

A non-interactive zero knowledge proof (NiZK) system allows a prover to generate proof for a statement. The verifier can learn the truth of the statement by verifying the proof, but cannot learn any information beyond that. We define a NiZK system for a relation R including two pairs of group elements $((g, U), (V, W))$ with equivalent discrete logs [19]. The relation is $R = \{(g, U, V, W), (\alpha) : U = g^\alpha \wedge W = V^\alpha\}$. A prover can use the NiZK system to generate proof that two pairs of group elements satisfy the relation R . The NiZK system consists of the following three algorithms. The setup algorithm takes a security parameter λ and outputs the public parameters $\text{pp} \leftarrow \text{NiZK.Setup}(\lambda)$. The proving algorithm takes a statement x and a witness w , outputs a proof $\pi \leftarrow \text{NiZK.Prove}_{\text{pp}}(w, x)$. The verification algorithm takes the statement x and the proof π as inputs, outputs a flag $b \leftarrow \text{NiZK.Vfy}_{\text{pp}}(x, \pi)$. If b is equal to 1, then π is valid; otherwise, π is invalid. A secure NiZK scheme has the following properties:

- **Completeness:** If a statement is true, then the prover with a witness can convince the verifier.
- **Knowledge Soundness:** The prover can only generate valid proofs for true statements.
- **Zero Knowledge:** The proof does not leak any information besides the truth of the statement.

In this paper, NiZK is used to implement the uniqueness of the POPRF in SADS-I. Note that we follow the constructions of NiZK in [19] and [39]. It achieves completeness, knowledge soundness, and zero knowledge in the random oracle model.

C. POPRF

POPRF [19], [20] extends the functionality of OPRF [24] to include a public input for the PRF evaluation. A client can learn and verify the evaluation $\text{PRF}_{sk}(t, x)$ for a public input t and a private input x . A POPRF scheme consists of the algorithms and protocol (POPRF.Setup , POPRF.KG , POPRF.Ev , POPRF.Vf). The setup algorithm

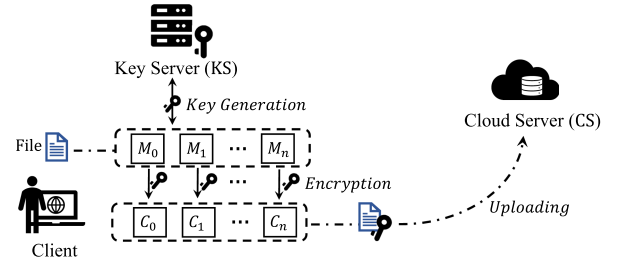


Fig. 4. Architecture of SADS-I.

takes a security parameter λ and outputs the public parameter $\text{pp} \leftarrow \text{POPRF.Setup}(\lambda)$. The key generation algorithm outputs the key pair $(pk, sk) \leftarrow \text{POPRF.KG}(\text{pp})$. During the evaluation protocol $(y, \varepsilon) \leftarrow \text{POPRF.Ev}(C(t, x), \text{KS}(sk))$, the client C inputs a public input t and a private input x , while the key server KS inputs the secret key sk . When the protocol concludes, the client learns the evaluation output $y = \text{PRF}_{sk}(t, x)$, while the key server learns nothing. The client can input the public key pk to run the verification algorithm $f \leftarrow \text{POPRF.Vf}(pk, t, x, y)$ to check whether y is valid. A secure POPRF has the following properties:

- **Pseudorandomness:** The evaluation output of POPRF is indistinguishable from random bit strings.
- **Request privacy:** The key server learns nothing about the private input x during the POPRF.
- **Uniqueness:** If the key server performs the evaluation using a secret key sk' that is not associated with the public key pk , the client can detect that and reject the output of the evaluation.

D. Intel SGX

SADS-II achieves similarity-aware rate limiting based on the trusted execution environment (called an *enclave*) provided with Intel SGX. Sensitive programs can run inside the enclave that resides in a hardware-guarded memory region called the *enclave page cache (EPC)*. Any outside application cannot access in-enclave contents or tamper with program execution inside the enclave. Additionally, SGX provides the function of *remote attestation*, which allows a remote entity to verify the integrity of the code running inside a target enclave. The limitation of Intel SGX is that the EPC size is limited, so we need to limit the size of in-enclave contents [29], [40], [41].

IV. SADS-I

In this section, we present SADS-I, which implements similarity-aware rate limiting based on cryptographic primitives. We first introduce its architecture and threat model, then present the similarity-aware rate-limiting strategy and the construction of SADS-I.

A. Architecture

As shown in Fig. 4, SADS-I follows the architecture of server-aided MLE, which consists of clients, a cloud server (CS for short), and a key server (KS for short). The client provides

an interface for a user to outsource data to CS. It divides user data into chunks, interacts with KS to obtain chunk keys, and then uploads encrypted chunks to CS. CS provides data storage services and implements cross-user deduplication to improve storage efficiency. KS is responsible for assisting clients in generating chunk keys to resist offline/online BFAs.

B. Threat Model

Our threat model assumes that the communications between clients and KS are encrypted and authenticated (e.g., using TLS). Besides, we focus on the defense against BFAs. The frequency analysis is beyond our scope and can be addressed by existing defense schemes [42], [43], [44]. SADS-I aims to provide confidentiality guarantees against the following adversaries:

- *Malicious client*: A malicious client manages to restore sensitive information in a target file F owned by a victim client. Suppose that the malicious client can obtain the ciphertext C_F of F and construct a dictionary of candidate files to launch online BFAs. Also, we assume that the malicious client can use multiple Sybil accounts to launch Sybil BFAs to try to bypass the rate limiting, which is a new adversary ability that has not been considered in existing server-aided MLE [13], [27], [45].
- *Honest-but-curious cloud server*: The cloud server CS honestly executes the assigned tasks but attempts to learn the underlying plaintext of outsourced data.
- *Honest-but-curious key server*: The key server KS is assumed to be honest-but-curious, as previous works did [13], [28], [42]. KS stores a secret key securely and honestly executes key generation with clients. But it tries to learn data information during the interactions with clients. Note that SADS-I leaks data similarity to KS. We discuss this point in Section IV-F in detail. We assume that KS cannot collude with CS or any clients, which follows the existing threat model [13], [28], [42]. Otherwise, the security will degrade to convergent security.

C. Design Challenges and Solutions

Here, we present the design challenges (abbreviated as $\mathcal{C}$) of SADS-I and our solutions.

C1: How to defend against Sybil BFAs while setting flexible rate-limiting thresholds? As stated in Section I, the existing rate-limiting strategy [13] is vulnerable to Sybil BFAs, and the setting of rate-limiting thresholds is improper. To this end, we propose a *similarity-aware rate-limiting strategy* that incorporates data similarity into key generation and rate limiting. In the similarity-aware rate-limiting strategy, during the key generation of a chunk, a client needs to input a chunk feature, which is used to represent data similarity. The rate limiting is enforced based on the number of requests corresponding to each feature, rather than the number of requests per client. This means that even if an adversary uses multiple Sybil accounts to make a large number of requests, the similarity-aware rate limiting can still provide a high delay to mitigate the attack since the feature corresponding to these requests is the same. Furthermore, we can set different rate-limiting thresholds for different features according to the

expected number of requests for them, resulting in more flexible and accurate threshold settings. We detail the threshold setting in Section VII-A.

C2: How to introduce data similarity in key generation?

The key generation in existing rate-limiting strategy [13] only involves data fingerprints, lacking the data features that represent data similarity. A straightforward approach is to directly combining data similarity with the existing OPRF-based server-aided MLE. Specifically, a client submits a pair of a chunk feature and a blinded chunk fingerprint to KS. Then, KS performs rate limiting based on the number of requests corresponding to features and runs the OPRF evaluation using the blinded fingerprint and the secret key. Subsequently, it returns the evaluation result to the client. The client generates keys based on the evaluation result. In this approach, the evaluation of OPRF only includes fingerprints without features. As a result, a malicious client may send inconsistent features and blinded fingerprints to KS in an attempt to evade rate limiting. For example, the blinded fingerprints sent by the malicious client are generated based on similar candidate data, but the features are arbitrarily constructed and are not the same. Then KS will be unable to detect online BFAs and cannot perform rate limiting.

To address the above issue, SADS-I utilizes *POPRF* [19], [20], [21] to implement similarity-aware rate limiting. During the key generation of a chunk, a client treats the chunk feature FT as a public input and the chunk fingerprint fp as a private input, and then uses (FT, fp) to run the POPRF with KS. Then, the client generates a server-aided MLE key k_M based on the POPRF evaluation output $\text{PRF}_{sk}(FT, fp)$, where sk denotes the secret key managed by KS. KS performs rate limiting based on the number of requests corresponding to features and sets a unique threshold for each feature, so the similarity-aware rate-limiting can be achieved. In SADS-I, the POPRF evaluation is run using both fingerprints and features. If a malicious client sends inconsistent fingerprints and features, it will be unable to generate a valid key to recover the target ciphertext. Note that the POPRF we leverage is proposed by Tyagi et al. [19] because of its high efficiency.

D. Construction of SADS-I

Here, we provide the detailed construction of SADS-I.

Initialization: The key server KS sets a security parameter λ and generates the public parameter pp , which consists of a large prime p , a multiplicative cyclic group $\mathbb{G}$ of prime order p , and a generator g of $\mathbb{G}$. The hash functions are $H_1 : \{0, 1\}^* \rightarrow \mathbb{G}$, $H_2 : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$, $H_3 : \{0, 1\}^* \rightarrow \mathbb{Z}_p$. KS generates its secret key $sk \in \mathbb{Z}_p$ and public key $pk \leftarrow g^{sk}$. KS stores sk securely and publishes $(pk, \text{pp} = \{p, \mathbb{G}, g, H_1, H_2, H_3\})$.

Chunking and feature generation: For an outsourced file F , the client C first partitions it into variable-size chunks based on Rabin fingerprinting [36]. Then, C computes a feature for each chunk to identify similar data. N-transform [30], [38] is a classical approach for generating data features. However, it generates multiple features for a single chunk. In order to combine multiple features into a single feature, we integrate the N-transform with similarity-preserving hash [37] to generate

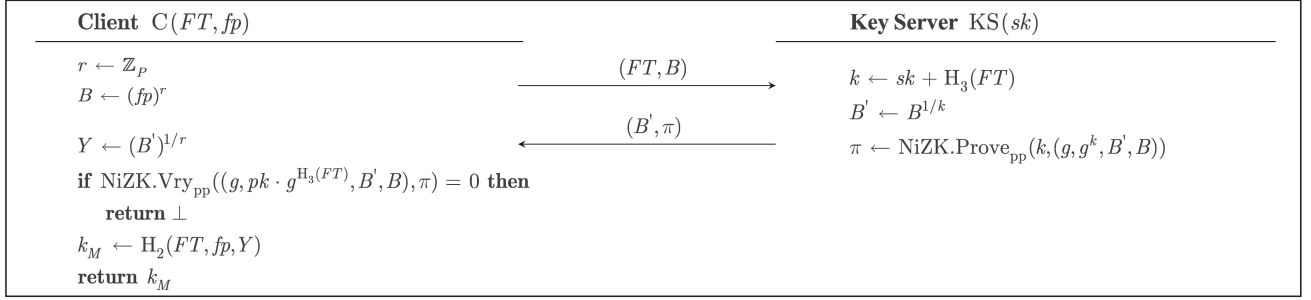


Fig. 5. Key generation in SADS-I.

TABLE I
RECORDS IN KS

Feature	Count	Threshold	Delay(s)
FT_0	15	235	0
FT_1^*	302	278	177
FT_2^*	168	100	177
...	...	...	...
FT_n	20	47	0

* The count of requests is greater than the threshold, so a delay for rate limiting is set for requests.

features. The specific steps of feature generation are as follows. First, we employ a rolling hash method [36] to the chunk, slide the window of rolling hash byte-by-byte, and collect the hash value of each window. Then, we calculate N linear transforms (details in [38]) on all hash values and record the minimum value of each linear transform as the sub-feature. Initialize a l -counter vector $V = (v_1, \dots, v_l)$ and compute a hash value for each sub-feature with a cryptographic hash function (e.g., SHA-256) [37]. Note that the length of the output of the hash function needs to be equal to l . If the i -th bit ($i \in \{1, \dots, l\}$) in the hash digest of a sub-feature is 0, then $v_i \leftarrow v_i - 1$; otherwise, $v_i \leftarrow v_i + 1$. After all N sub-features have been hashed into V , for all $i \in \{1, \dots, l\}$, if $v_i > 0$, set v_i to 1; otherwise, set v_i to 0. The resulting vector V is set to the feature of the chunk.

Key generation and data upload: We describe the process of key generation and data upload with a chunk M as an example. The client C first computes a fingerprint $fp \leftarrow H_1(M)$ and a feature FT for M . Then, it uses (FT, fp) to interact with KS to generate a key for M (see Fig. 5). Specifically, C sends a blinded fingerprint $B \leftarrow (fp)^r$ and feature FT to KS . The latter computes $k \leftarrow sk + H_3(FT)$, and then returns $B' \leftarrow B^{1/k}$ and a NiZK proof π to C . The trick of the proof is as follows. KS computes a proof that $B = (B')^k$, while C performs the verification by explicitly reconstructing $g^k = pk \cdot g^{H_3(FT)} = g^{sk+H_3(FT)}$ [19]. The relation of NiZK in our scheme is $R = \{(g, g^k, B', B)(k)\}$.

Subsequently, KS retrieves FT in its records (see Table I). If FT has not been stored, KS records FT and sets its count to 1. Otherwise, it increments the existing count of FT by 1. In case the current count exceeds the threshold, KS performs rate limiting based on the delay corresponding to FT . The setting of the delay and threshold is introduced in Section VII-A. Upon

receiving B' , C removes the blinding to get $Y = (B')^{1/r}$ and verifies the validity of Y by running the NiZK verification algorithm. If the verification algorithm returns false, C discards Y . Otherwise, C generates the key $k_M \leftarrow H_2(FT, fp, Y)$ for M . During the data upload, C computes the encrypted chunk $C_M \leftarrow \text{Enc}_{SE}(k_M, M)$ and tag $T_M \leftarrow H_2(C_M)$, uploads C_M to CS , and stores (k_M, T_M) locally. CS computes tag $T_M \leftarrow H_2(C_M)$ and then checks whether it has been stored previously. If so, C_M is a duplicate chunk, and CS does not store it. Otherwise, CS stores (C_M, T_M) . Note that we let CS compute T_M to prevent inconsistency between tags and encrypted chunks [12].

Data download: To restore chunk M , C sends tag T_M to CS . The latter returns the encrypted chunk C_M . C checks whether $H_2(C_M)$ is equal to T_M to verify integrity and then restores chunk $M \leftarrow \text{Dec}_{SE}(k_M, C_M)$.

Rate limiting: As shown in Table I, KS records features, their thresholds, the count of requests corresponding to them, and their corresponding delays. KS sets a unique threshold for each feature. When the number of requests for a feature exceeds its threshold (e.g., FT_1 and FT_2 in Table I), KS performs rate limiting based on the delay corresponding to the feature to defend against potential online BFAs. Similar to the existing rate-limiting strategy [13], our similarity-aware rate limiting evolves over epochs. When an epoch ends, KS clears all features, and the count is restarted in the new epoch, thus KS does not have too much memory overhead incurred by the rate limiting.

E. Criteria for Feature Definition

In our scheme, features are used to represent the bit patterns of data and identify similar data chunks. The definition of features needs to satisfy the following criteria:

- **Similarity identification capability:** Features need to accurately identify the similarity of data chunks, ensuring that similar data chunks have the same feature, and dissimilar data chunks have different features. In terms of implementation, we adopt the existing N-transform and similarity-preserving hash technologies to extract data features. Existing works [30], [37] have verified that these methods can satisfy similarity identification capability.
- **Computational efficiency:** Data features need to be extracted based on the plaintext data content. To prevent data privacy leakage, it is necessary for clients to generate data features locally [37], [46]. This requires feature generation

and tag $T_M \leftarrow H(C_M)$, uploads C_M to the cloud server CS, and stores (k_M, T_M) locally. CS computes the tag $T_M \leftarrow H(C_M)$ and deduplicates C_M , which is similar to the process in SADS-I. The data download step remains the same as in SADS-I, so we omit it.

Rate limiting: As shown in Table II, since the enclave learns features, fingerprints, and the counting information of each feature, SADS-II is capable of achieving similarity-aware rate limiting while identifying requests for duplicate data. Requests for duplicate chunks contain the same fingerprint, allowing the enclave to identify them by deduplicating the fingerprints in requests and only store unique fingerprints. KS performs rate limiting based on the number of unique fingerprints corresponding to features. Consequently, the enclave records only once for multiple requests for duplicate chunks, addressing the limitation of SADS-I and the existing rate-limiting strategy [13].

Metadata management: In SADS-I and SADS-II, the client needs to store chunk keys and tags as metadata. When dealing with large-scale datasets, storing these metadata on the client may incur significant storage overhead. The metadata management issue can be addressed by outsourcing encrypted metadata to the cloud server. The client can encrypt chunk keys and tags using its master key, then outsource the encrypted keys and tags to the cloud server to reduce local storage burden. Furthermore, existing metadata deduplication schemes [49] can be integrated to minimize metadata storage costs on the cloud server.

Memory overhead in enclaves: The limited memory space of enclaves may be a key issue restricting the deployment of SADS-II in large-scale systems. This issue can be solved through the following two methods. First, the rate limiting in SADS-II also evolves over epochs. At the end of each epoch, the enclave clears its records, ensuring that the size of in-enclave contents does not become too large. Second, previous works [50], [51], [52] have demonstrated that in real datasets, a large number of data chunks correspond to a small fraction of features. That is to say, the usage frequency of data features is uneven. Therefore, similar to DEBE [40], we can only record the most frequently used features in the enclave (using the least-frequently-used (LFU) eviction policy) to alleviate the memory overhead of the enclave. Features with low usage frequency can be encrypted and stored outside the enclave. After receiving a feature, the enclave first retrieves it from its internal index of high-frequency features. If the feature cannot be retrieved, the enclave will then read low-frequency-used features from external sources for retrieval. Due to the uneven usage frequency of features, the enclave can retrieve the received feature internally with a high probability. This method can alleviate the EPC paging overhead and reduce the cost of context switching.

C. Discussion

Here, we discuss some potential issues regarding our similarity-aware rate limiting.

Slow attack: Similar to the existing rate-limiting strategy [13], both SADS-I and SADS-II implement cross-epoch rate limiting. At the end of each epoch, the number of requests for key generation is cleared. In this case, the adversary may launch the

slow attack. That is, the adversary issues requests over multiple epochs to reduce the delay introduced by the rate limiting. Nevertheless, this attack is very inefficient because the adversary can only obtain a very limited number of keys at each epoch. As a result, it takes a very long time for the adversary to get all the candidate keys. We can set an appropriate epoch length and threshold to mitigate the slow attack.

Impact on the performance of uploading popular data: When uploading popular data (uploaded by many users), it is possible that a large number of data correspond to the same feature, resulting in the execution of rate limiting to reduce performance. We set reasonable rate-limiting thresholds for features to solve this issue. The threshold is set to the expected number of requests for that feature plus a random buffer [13]. The expected number of requests is determined based on the dataset, and the size of the buffer can be self-configured. For example, for a feature, we can set its threshold as the expected amount of requests when the system is operating normally. Suppose FT_p is the feature corresponding to the popular data. FT_p has a high expected number of requests, so its threshold is large. Even if the number of requests for it is relatively large, as long as the number is below the threshold, rate limiting will not be triggered. As a result, through the reasonable setting of thresholds, our similarity-aware rate limiting does not degrade the performance of uploading popular data.

Impact on legitimate users: The similarity-aware rate limiting is a global-scale rate limiting mechanism. It may degrade the key generation performance of legitimate users while adding delay to the adversary. In our implementation (Section VII), the rate limiting is performed only if the number of key generation requests corresponding to a feature is greater than its threshold. Meanwhile, in real-world datasets, it is unlikely that a large number of chunks correspond to the same feature (see Section III). Assume that the feature of the target chunk of the adversary is FT_i . As long as the feature of the chunk uploaded by the legitimate user is different from FT_i , the performance will not be decreased by the rate limiting. If the feature of the chunk uploaded by the user is equal to FT_i , the user will be rate-limited by the key server when performing key generation for the chunk. If the user perceives the rate limit, it indicates that the chunk corresponding to this feature may be under BFAs. The user can choose to randomly generate a symmetric key locally to encrypt the chunk. Although this approach slightly decreases deduplication ratio, the chunks of legitimate users are strongly secured and users do not have to endure high delay.

SADS-I and SADS-II: SADS-II protects data features and demonstrates excellent performance, while SADS-I leaks data features to KS. However, as discussed in [46], it is infeasible for the adversary to learn data information based on data features. First, since a feature can match many chunks, the adversary cannot launch BFAs based on the feature to learn specific plaintext information. Besides, even if the adversary conducts statistical analysis on the features, it can only learn which key generation requests in an epoch correspond to similar data, and cannot obtain further information. Compared to SADS-II, the advantage of SADS-I lies in its elimination of the need to introduce trusted hardware, resulting in lower deployment costs.

Number of features and key generation: Our scheme uses a single feature to identify similar data chunks. A potential issue is that only highly similar chunks may be recognized as similar ones. In fact, the key generation method in our schemes can be extended to use multiple features, and the key generation step remains unchanged. Clients can generate multiple features for data chunks and send them to the key server for key generation. Clients obtain the same key only if the data chunks are completely identical. Thus, whether a client sends one feature or multiple features to the key server, it will not affect the key obtained by the client. In addition, even if only one feature is used for key generation, the detection of similar chunks still can be relaxed. For example, the MinHash algorithm [28] can be used to generate data features. In this case, more data chunks can be identified as similar. We use a single feature to detect data similarity in order to save the system's memory overhead.

Comparison with FeatureSpy: Similar to our schemes, FeatureSpy [30] also defends against security threats in deduplicated storage systems based on data features. However, there are the following fundamental differences between our schemes and FeatureSpy:

- *Security objectives:* FeatureSpy defends against side-channel attacks, where adversaries exploit duplicate responses to infer data information. Our schemes resist BFAs, where adversaries interact with the key server to enumerate all possible keys to recover data information. These two attack paths are fundamentally distinct. Moreover, our schemes provide semantic security for user data. FeatureSpy retains data similarity in ciphertexts and offers only convergent security.
- *Defense methods:* Our schemes integrate data similarity into key generation and rate limiting, slowing down Sybil BFAs by imposing similarity-aware rate limiting. FeatureSpy blocks side-channel attacks by directly terminating the data upload process. Unlike side-channel attacks, BFAs cannot be mitigated by simple terminating upload process; effective rate limiting is essential. Thus, although both schemes consider data similarity, our core innovation lies in designing efficient rate-limiting strategies, which FeatureSpy does not address.

VI. SECURITY ANALYSIS

In this section, we analyze the security of our proposed schemes based on our threat model (see Section IV-B). Here, we assume that online/Sybil BFAs can be defended against with similarity-aware rate limiting. The defense effectiveness of the rate limiting is presented in Section VII-B.

A. Security of SADS-I

We first analyze the security of the key generation in SADS-I. Specifically, we prove that the client is unable to learn the secret key sk of KS and can verify the validity of the evaluation results. Meanwhile, KS cannot obtain specific data information during the interaction with clients. Then, a proof is provided that the honest-but-curious CS is unable to recover data information from outsourced data. Additionally, it is proven that SADS-I provides resistance to malicious clients.

Theorem 1: Assume that the POPRF satisfies pseudorandomness, request privacy, and uniqueness. Then, the key generation in SADS-I is secure and verifiable.

Proof: Since POPRF provides *pseudorandomness*, the evaluation result $\text{PRF}_{sk}(FT, fp)$ is indistinguishable from random bit strings. As a result, the client cannot learn any information about the secret key sk during the interaction with KS. Similarly, based on the *request privacy* of POPRF, the private input fp of the client remains secret from KS. The only information leaked to KS is the public input FT . In other words, KS can only use data features to obtain the similarity of the data corresponding to the key generation requests, but it cannot learn the data hash. Therefore, it cannot launch offline BFAs to restore data information. Moreover, the *uniqueness* of POPRF ensures the verifiability of the key generation. Upon receiving (B', π) sent by KS, the client can run the verification algorithm of POPRF to verify the validity of the POPRF output. If KS tricks clients into accepting inconsistent evaluations, then it breaks the uniqueness of POPRF. $\square$

Theorem 2: Assume that the symmetric encryption provides semantic security. Then, in SADS-I, the honest-but-curious cloud server cannot learn data information from outsourced data.

Proof: The data accessible to the cloud server CS are the symmetric ciphertexts of the data chunks. If CS recovers data information from the encrypted chunks, then it compromises the semantic security of symmetric encryption and breaks our assumption. Additionally, as the symmetric key incorporates the secret key sk managed by KS, CS cannot obtain any valid keys through offline BFAs. $\square$

Security against malicious clients: According to Theorem 1, the malicious client is unable to learn sk during the key generation. Consequently, just like CS, the malicious client cannot launch offline BFAs on the encrypted data. In the event that the malicious client launches Sybil BFAs, SADS-I is capable of utilizing similarity-aware rate limiting to counteract the attacks.

Remarks: In the worst-case scenario where the secret key sk is compromised, the security of SADS-I degrades to convergent security, as in the existing server-aided MLE [13]. Additionally, SADS-I leaks features to KS. But KS cannot map features to specific chunks since many chunks correspond to the same feature. Hence, we believe that this relaxation of security is acceptable.

B. Security of SADS-II

The security of SADS-II relies on the security of the enclave and symmetric encryption. The enclave provides a trusted execution environment that prevents external access, ensuring that KS cannot access the master key msk , data features, and fingerprint stored within it. Consequently, from KS's perspective, it receives encrypted features and fingerprints from clients, as well as encrypted keys from the enclave. If KS is able to restore data information from these symmetric ciphertexts, it would compromise the semantic security provided by the symmetric encryption.

Since the symmetric key k_M and master key msk are secret for CS, it can neither recover any data information based on outsourced symmetric ciphertexts nor obtain any valid keys

through offline BFAs. Furthermore, since the cryptographic hash function H is unidirectional, the malicious client cannot learn msk from k_M . Therefore, it cannot obtain any valid keys through offline BFAs. Online BFAs can be mitigated by the similarity-aware rate limiting.

VII. EVALUATION

We implement the prototypes of our schemes and conduct a series of experiments for evaluation. The prototypes are developed in C++ using OpenSSL [53] and consist of the client, the cloud server, and the key server. The POPRF in SADS-I is implemented based on the OpenSSL library [53]. The trusted execution environment in SADS-II is implemented using Intel SGX SDK 2.7 [54] and Intel SGX SSL [55]. The hash functions and symmetric encryption are implemented by SHA-256 and AES-256, respectively. We also implement the state-of-the-art server-aided MLE scheme, *DupLESS* [13], for performance comparison. Our experiments run on a machine equipped with a quad-core 3.4 GHz Intel Core i5-9300 CPU, a 1 TB 5400 RPM SATA hard disk, and 8 GB RAM. All evaluation results are averages of 50 runs. We release the source code of our prototypes at: <https://github.com/Obscure12138/SADS>.

Section VII-B evaluates the delay incurred by our similarity-aware rate limiting and the existing per-client rate limiting [13] when an adversary launches Sybil BFAs. Since the existing rate limiting cannot identify requests for duplicate data, we employ the similarity-aware rate limiting in SADS-I to ensure a fair evaluation. Also, Section VII-B compares the rate limiting difference between SADS-I and SADS-II. In Section VII-C, we use both synthetic data and real-world data to evaluate the data upload performance of SADS-I, SADS-II, and *DupLESS* [13]. The *Enron* dataset [31] is used in Section VII-B as it provides data content for computing features. It allows us to evaluate the delay incurred by rate limiting. In Section VII-C, the *FSL* dataset [56] is employed. Although it solely provides data fingerprints, its data volume is relatively large, and it is often used in the performance evaluation for encrypted deduplication schemes [15], [26], [28], [29]. We summarize the evaluation results as follows.

- When an adversary launches Sybil BFAs, our similarity-aware rate limiting can provide a stable and high delay for defense. However, the delay provided by the existing per-client rate limiting decreases with the increase of the number of Sybil accounts.
- When the number of duplicate chunks corresponding to the same feature in an epoch is large, the delay introduced by SADS-II is significantly lower than that introduced by SADS-I. Note that the reduced delay does not affect the defense of online BFAs.
- The computation overhead of SADS-I is slightly higher than that of the existing server-aided MLE [13] because it introduces POPRF and feature generation. SADS-II has significantly better performance due to its utilization of SGX avoiding expensive cryptographic operations.

A. Setting of Thresholds and Delays

Before presenting the evaluation results, we first introduce the setting of thresholds and delays. We take a feature FT_i as an example. As advised in [13], the threshold should be set as the expected number of requests plus a buffer. We define the number of chunks corresponding to FT_i in the dataset as N_i and consider it as the expected number of requests. In addition, the key server sets a random buffer for each feature at each epoch. If the buffer corresponding to FT_i is Br_i , then its rate-limiting threshold q_i is set to $(N_i + Br_i)$.

We use the *count* of a feature to denote the number of requests corresponding to it. When the count ctr_i of FT_i exceeds the threshold q_i , the rate limiting is performed. That is, the key server imposes a delay de_i for each key generation request corresponding to FT_i . Note that the key server processes requests serially, handling the next request only after finishing the previous one. The setting of de_i is as follows. Assume that the dataset contains a maximum of N_{max} chunks corresponding to the same feature. For feature FT_i , we set the upper bound on the count for it in an epoch to $(N_{max} + q_i)$. The setting of de_i needs to ensure that $(N_{max} + q_i)$ requests corresponding to FT_i can be handled within an epoch. Since the rate limiting is only performed when $ctr_i > q_i$, we allow N_{max} requests corresponding to FT_i can be handled after the rate limiting is performed. Therefore, we set de_i as t_E/N_{max} , where t_E denotes the epoch duration. Note that the delay for each feature is the same, which is to prevent the adversary from obtaining data information based on the difference in delay.

Remarks: A potential attack is that an adversary may infer data information by observing the delays imposed by the key server. The delays corresponding to all features are the same, and once the number of requests exceeds the threshold, the delay corresponding to each request is the same. Therefore, the adversary cannot obtain data information based on the difference of delay. In addition, since the key server sets a random buffer for each feature in each epoch, the adversary cannot learn the specific threshold corresponding to the feature. Even if the adversary observes that a certain request has been delayed by the key server, it cannot determine whether the delay is caused by its own number of requests exceeding the threshold or by other users uploading requests corresponding to that feature. Consequently, the adversary cannot obtain the data information of other users by observing whether the request is delayed or not.

B. Evaluation for the Delay Incurred by the Rate Limiting

We first compare the delay incurred by our similarity-aware rate limiting with the existing per-client rate limiting method [13] when an adversary launches online BFAs. Here, we consider the case of Sybil BFAs, where the adversary is capable of creating any number of Sybil accounts to launch online BFAs. Then, we compare the rate limiting difference between SADS-I and SADS-II.

Similarity-aware rate limiting versus per-client rate limiting: We divide the files in the *Enron* dataset [31] into chunks and compute a feature for each chunk. We keep track of all features

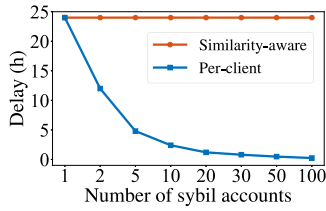


Fig. 7. Comparison of the similarity-aware rate limiting and per-client rate limiting.

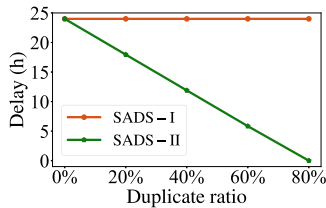


Fig. 8. Comparison of the rate limiting in SADS-I and SADS-II.

and the number of chunks corresponding to them. In total, there are 556,627 chunks and 384,814 features. The maximum number N_{max} of chunks corresponding to the same feature is 383. The epoch length t_E is set to 24 hours. The settings of thresholds and delays in both rate-limiting methods are the same. We evaluate the delay when the adversary employs different numbers of Sybil accounts to launch Sybil BFAs. Assume the feature of the adversary's target chunk is FT_i . For the convenience of recording delays, assume that legitimate users have uploaded q_i requests corresponding to FT_i , and the adversary initiates N_{max} requests to obtain as many keys as possible within an epoch. As illustrated in Fig. 7, the delay of our similarity-aware rate limiting remains constant irrespective of the number of Sybil accounts. The reason is that we perform rate limiting based on the number of requests for each feature rather than the number of requests per client. On the other hand, the delay of the per-client rate limiting decreases as the number of Sybil accounts increases. This is because the number of requests per Sybil account decreases. For example, when 10 Sybil accounts are used to upload 100 candidate chunks, the number of requests per Sybil account is only 10. The delay incurred by the per-client rate limiting depends on the number of requests per account.

SADS-I versus SADS-II: Although both SADS-I and SADS-II achieve similarity-aware rate limiting, SADS-II has the additional capability to identify key generation requests for duplicate chunks, while SADS-I lacks. Here, we contrast the rate limiting effects of SADS-I and SADS-II. We select a feature FT_i from the Enron dataset. Assume that q_i requests corresponding to FT_i have already been handled, then requests for N_{max} chunks are issued to the key server. We record the delay imposed by the key server for each request. We vary the duplicate ratio of the uploaded N_{max} chunks to 0%, 20%, 40%, 60%, and 80%, respectively. Fig. 8 presents the comparison results of the rate limiting in SADS-I and SADS-II. In SADS-I, the duplicate ratio has no effect on the delay since it cannot identify requests for duplicate chunks. In SADS-II, the delay decreases as the duplicate ratio increases. This is because SADS-II performs rate

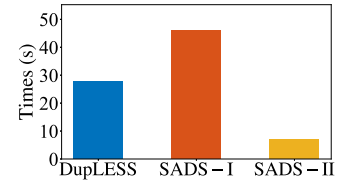


Fig. 9. Time overhead of uploading a 1 GB test file.

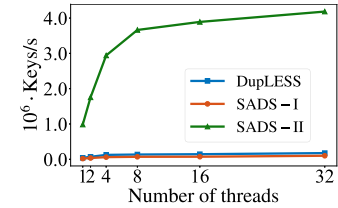


Fig. 10. Number of requests handled during multi-client key generation.

limiting based on the number of unique chunks corresponding to FT_i . For example, if the key server performs key generation request for 100 chunks and the duplicate ratio of these chunks is 80%, the number of requests for unique chunks reduces to 20, resulting in a significant decrease in the delay. It is important to note that the decrease of the delay in SADS-II does not weaken the defense against online BFAs, as requests for duplicate chunks do not allow the adversary to learn more keys.

C. Performance Evaluation for Data Upload

Here, we evaluate the data upload performance using both synthetic data and real-world data.

Synthetic data: We generate a 1 GB test file with random contents as the synthetic data. We evaluate the time overhead when uploading the test file. The upload steps include content-defined chunking, feature generation (only in our schemes), key generation, and data communication. We set the minimum, average, and maximum chunk sizes to 4 KB, 8 KB, and 16 KB, respectively.

First, we evaluate the time overhead of each scheme when a client uploads the 1 GB test file. As shown in Fig. 9, SADS-I has the highest time overhead, since it introduces POPRF and feature generation compared to DupLESS [13]. SADS-II has the lowest time overhead since it leverages Intel SGX rather than cryptographic primitives to implement encrypted deduplication. The time overhead of SADS-II is only 16.3% and 26.7% of that of SADS-I and DupLESS, respectively.

Then, to evaluate scalability, we use Google benchmark [57] to evaluate the number of key generation requests the key server could handle per second during multi-client key generation. We run multiple threads, where each thread simulates a client and initiates key generation requests to the key server. We denote cnt as the number of key generation requests the key server can handle per second. As shown in Fig. 10, the cnt of SADS-II is significantly higher than those of SADS-I and DupLESS due to its higher computation efficiency. When the number of threads is 32, the cnt of SADS-II is 23.9 and 43.0 times that of SADS-I and DupLESS, respectively. Compared to DupLESS, the cnt of SADS-I is slightly lower because of its marginally higher

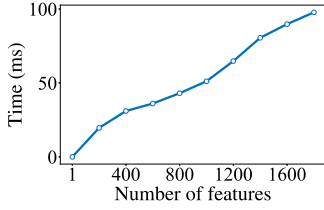


Fig. 11. Time overhead for feature generation.

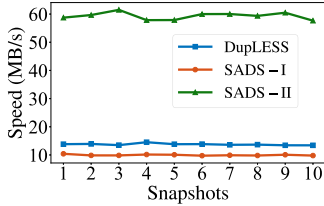


Fig. 12. Upload speed on the FSL dataset.

computation overhead. Additionally, as the number of threads increases, the growth rate of *cnt* for each scheme decreases, which is attributed to resource contention among threads and the increased time cost of thread switching.

Finally, we evaluate the computation overhead of feature generation. Fig. 11 shows the computation overhead of generating features for 1,600 chunks. Our feature generation is efficient, taking only 89.7 ms to generate features for 1,600 chunks. Note that due to the use of the content-defined chunking, the feature generation cost varies for chunks of different sizes. Thus, the time overhead is not exactly proportional to the number of features in Fig. 11.

Real-world data: We use the FSL dataset [56] to evaluate the data upload performance. The FSL dataset, collected by the File systems and Storage Lab (FSL) at Stony Brook University [56], contains the home directories snapshots of nine users. These snapshots only provide fingerprints of data chunks without the data content. To reconstruct each chunk, we repeatedly write its fingerprint into a spare chunk until it reaches the specified chunk size [28], [29], [40], [42]. Consequently, the same fingerprint in the FSL dataset can correspond to the same chunk in the reconstructed data. For our evaluation, we select the snapshots from September 1st to September 10 (2014) of a user. The pre-deduplicated data size is 8.24 GB, and the duplicate ratio is 85.7%. As shown in Fig. 12, since fewer cryptographic operations are performed, SADS-II achieves $5.64\text{--}6.24\times$ and $3.98\text{--}4.56\times$ speedups over SADS-I and DupLESS, respectively. Due to the introduction of POPRF and feature generation, SADS-I incurs a slowdown of 25%–30% in upload speed compared to DupLESS.

VIII. RELATED WORKS

MLE [12] achieves efficient encrypted deduplication by generating keys based on data content. Following the introduction of MLE, several encrypted deduplication schemes based on MLE have been proposed. Adabi et al. [58] strengthen the security notation in [12] by considering plaintext distributions that could potentially depend on the public parameters. Subsequently, μ R-MLE2 was introduced to improve the performance of the scheme

presented in [58]. Both BL-MLE [59] and MetaDedup [49] addressed the issue of metadata management in MLE. Liu et al. [60] utilize MLE to design an encrypted deduplication scheme that supports integrity auditing. Zhao et al. [61] propose updatable block-level MLE to achieve efficient updates in encrypted deduplication. Furthermore, Liu et al. [62] propose MLSE based on MLE, which enables encrypted deduplication as well as search operations on encrypted content.

To resist the offline BFAs for MLE, server-aided MLE [13] was proposed. It introduces a third-party key server to assist the client in key generation and prevents offline BFAs. The key server can implement rate limiting as a countermeasure against online BFAs. Some studies [25], [27], [45] have identified the potential issues of the third-party key server, such as performance bottlenecks and single points of failure. Liu et al. [25] utilize PAKE to realize encrypted deduplication without additional independent servers. However, their scheme entails significant computation overhead and necessitates frequent interactions among clients. Consequently, most of the existing encrypted deduplication schemes capable of withstanding BFAs still rely on server-aided MLE. Shin et al. [45] proposed decentralized server-aided encryption to provide multi-tenancy and scalability for the original server-aided MLE. SPADE [3] was proposed to periodically substitute key servers with newly employed ones in server-aided MLE. REED [28] was proposed to offer lightweight rekeying for server-aided MLE. Ha et al. [63] accomplished the key update for the master key in server-aided MLE. Fu et al. [64] implemented application-aware encrypted deduplication for Ehealth data using server-aided MLE.

Currently, almost all the existing encrypted deduplication schemes depend on the rate limiting enforced by the key server in the server-aided MLE to mitigate BFAs. As discussed in Section I, the existing rate-limiting strategy suffers from certain limitations. We propose the similarity-aware rate limiting for the first time as a solution to overcome these limitations.

IX. CONCLUSION

In this paper, we propose a similarity-aware rate limiting strategy to address the security threat of online BFAs in encrypted deduplication. Then, we present two similarity-aware defense schemes, SADS-I and SADS-II. SADS-I achieves similarity-aware rate limiting by leveraging POPRF, while SADS-II employs Intel SGX to protect data similarity, identify requests for duplicate data, and improve performance. Evaluation results demonstrate that our similarity-aware rate limiting provides a better defense effect than the existing rate-limiting strategy and offers high data upload performance. Overall, our proposed method proves to be secure and effective in mitigating online BFAs.

REFERENCES

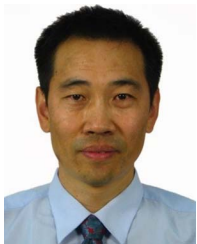
- [1] IDC, "Data age," 2025. [Online]. Available: <https://www.seagate.com/our-story/data-age-2025/>
- [2] W. Susilo, P. Jiang, J. Lai, F. Guo, G. Yang, and R. H. Deng, "Sanitizable access control system for secure cloud storage against malicious data publishers," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 3, pp. 2138–2148, May/Jun. 2022.

- [3] Y. Zhang, C. Xu, N. Cheng, and X. Shen, "Secure password-protected encryption key for deduplicated cloud storage systems," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2789–2806, Jul./Aug. 2022.
- [4] Y. Miao, Q. Huang, M. Xiao, and W. Susilo, "Blockchain assisted multi-copy provable data possession with faults localization in multi-cloud storage," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 3663–3676, 2022.
- [5] K. Zhang, Z. Jiang, J. Ning, and X. Huang, "Subversion-resistant and consistent attribute-based keyword search for secure cloud storage," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 1771–1784, 2022.
- [6] Dropbox, (n.d.). 2024. [Online]. Available: <http://www.dropbox.com/>
- [7] Mozy, (n.d.). 2024. [Online]. Available: <http://mozy.com/>
- [8] Google drive, (n.d.). 2024. [Online]. Available: <http://drive.google.com>
- [9] W. Xia et al., "A comprehensive study of the past, present, and future of data deduplication," *Proc. IEEE*, vol. 104, no. 9, pp. 1681–1710, Sep. 2016.
- [10] D. T. Meyer and W. J. Bolosky, "A study of practical deduplication," *ACM Trans. Storage*, vol. 7, no. 4, pp. 14:1–14:20, 2012.
- [11] G. Wallace et al., "Characteristics of backup workloads in production systems," in *Proc. 10th USENIX Conf. File Storage Technol.*, 2012, Art. no. 4.
- [12] M. Bellare and S. Keelveedhi, *Message-Locked Encryption and Secure Deduplication*. Berlin, Germany: Springer, 2013.
- [13] M. Bellare, S. Keelveedhi, and T. Ristenpart, "DupLESS: Server-aided encryption for deduplicated storage," in *Proc. Usenix Conf. Secur.*, 2013, pp. 179–194.
- [14] D. Harnik, B. Pinkas, and A. Shulman-Peleg, "Side channels in cloud services: Deduplication in cloud storage," *IEEE Secur. Privacy*, vol. 8, no. 6, pp. 40–47, Nov./Dec. 2010.
- [15] P. Zuo et al., "Mitigating traffic-based side channel attacks in bandwidth-efficient cloud storage," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2018, pp. 1153–1162.
- [16] C. Yu, S. P. Gochhayat, M. Conti, and C. Lu, "Privacy aware data deduplication for side channel in cloud storage," *IEEE Trans. Cloud Comput.*, vol. 8, no. 2, pp. 597–609, Second Quarter 2020.
- [17] Z. Pooranian, K.-C. Chen, C.-M. Yu, and M. Conti, "RARE: Defeating side channels based on data-deduplication in cloud storage," in *Proc. 2018 IEEE Conf. Comput. Commun. Workshops*, 2018, pp. 444–449.
- [18] J. Stanek and L. Kencl, "Enhanced secure thresholded data deduplication scheme for cloud storage," *IEEE Trans. Dependable Secure Comput.*, vol. 15, no. 4, pp. 694–707, Jul./Aug. 2018.
- [19] N. Tyagi, S. Celi, T. Ristenpart, N. Sullivan, S. Tessaro, and C. A. Wood, "A fast and simple partially oblivious PRF, with applications," in *Proc. 41st Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2022, pp. 674–705.
- [20] A. Everspaugh, R. Chatterjee, S. Scott, A. Juels, and T. Ristenpart, "The Pythia PRF service," in *Proc. 24th USENIX Secur. Symp.*, 2015, pp. 547–562.
- [21] S. Jarecki, H. Krawczyk, and J. K. Resch, "Threshold partially-oblivious PRFs with applications to key management," *IACR Cryptol. ePrint Arch.*, 2018, Art. no. 733.
- [22] Y. Shin, D. Koo, and J. Hur, "A survey of secure data deduplication schemes for cloud storage systems," *ACM Comput. Surv.*, vol. 49, no. 4, pp. 74:1–74:38, 2017.
- [23] J. R. Douceur, A. Adya, W. J. Bolosky, P. Simon, and M. Theimer, "Reclaiming space from duplicate files in a serverless distributed file system," in *Proc. 22nd Int. Conf. Distrib. Comput. Syst.*, 2002, Art. no. 617.
- [24] M. Naor and O. Reingold, "Number-theoretic constructions of efficient pseudo-random functions," *J. ACM*, vol. 51, no. 2, pp. 231–262, 2004.
- [25] J. Liu, N. Asokan, and B. Pinkas, "Secure deduplication of encrypted data without additional independent servers," in *Proc. 22nd ACM SIGSAC Conf. Comput. Commun. Secur.*, 2015, pp. 874–885.
- [26] Y. Zhou et al., "SecDep: A user-aware efficient fine-grained secure deduplication scheme with multi-level key management," in *Proc. IEEE 31st Symp. Mass Storage Syst. Technol.*, 2015, pp. 1–14.
- [27] M. Miao, J. Wang, H. Li, and X. Chen, "Secure multi-server-aided data deduplication in cloud computing," *Pervasive Mobile Comput.*, vol. 24, pp. 129–137, 2015.
- [28] C. Qin, J. Li, and P. P. C. Lee, "The design and implementation of a rekeying-aware encrypted deduplication storage system," *ACM Trans. Storage*, vol. 13, no. 1, pp. 9:1–9:30, 2017.
- [29] Y. Ren, J. Li, Z. Yang, P. P. Lee, and X. Zhang, "Accelerating encrypted deduplication via SGX," in *Proc. USENIX Annu. Tech. Conf.*, 2021, pp. 957–971.
- [30] J. Li, Y. Ren, P. P. C. Lee, Y. Wang, T. Chen, and X. Zhang, "FeatureSpy: Detecting learning-content attacks via feature inspection in secure deduplicated storage," in *Proc. IEEE Conf. Comput. Commun.*, 2023, pp. 1–10.
- [31] Enron mail dataset, (n.d.). 2025. [Online]. Available: <https://www.cs.cmu.edu/~enron/>
- [32] S. He, J. Zhu, P. He, and M. R. Lyu, "Loghub: A large collection of system log datasets towards automated log analytics," 2020, *arXiv: 2008.06448*.
- [33] The European climate assessment & dataset project, (n.d.). 2025. [Online]. Available: <https://www.ecad.eu>
- [34] GCC, "The GNU compiler collection," (n.d.). 2025. [Online]. Available: <https://gcc.gnu.org/>
- [35] GDB, "The GNU project debugger," (n.d.). 2025. [Online]. Available: <https://sourceware.org/gdb/>
- [36] M. O. Rabin, "Fingerprinting by random polynomials," *Tech. Rep.*, 1981.
- [37] T. Jiang et al., "FuzzyDedup: Secure fuzzy deduplication for cloud storage," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 3, pp. 2466–2483, May/Jun. 2023.
- [38] P. Shilane, M. Huang, G. Wallace, and W. Hsu, "WAN optimized replication of backup datasets using stream-informed delta compression," in *Proc. 10th USENIX Conf. File Storage Technol.*, 2012, Art. no. 5.
- [39] D. Chaum and T. P. Pedersen, "Wallet databases with observers," in *Proc. Annu. Int. Cryptol. Conf.*, 1992, pp. 89–105.
- [40] Z. Yang, J. Li, and P. P. Lee, "Secure and lightweight deduplicated storage via shielded deduplication-before-encryption," in *Proc. USENIX Annu. Tech. Conf.*, 2022, pp. 37–52.
- [41] G. Ha, X. Ge, C. Jia, Y. Chen, and Z. Su, "Revisiting SGX-based encrypted deduplication via pow-before-encryption and eliminating redundant computations," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 3, pp. 2037–2053, May/Jun. 2025.
- [42] J. Li, Z. Yang, Y. Ren, P. P. Lee, and X. Zhang, "Balancing storage efficiency and data confidentiality with tunable encrypted deduplication," in *Proc. 15th Eur. Conf. Comput. Syst.*, 2020, pp. 1–15.
- [43] J. Li, G. Wei, J. Liang, Y. Ren, P. P. Lee, and X. Zhang, "Revisiting frequency analysis against encrypted deduplication via statistical distribution," in *Proc. IEEE Conf. Comput. Commun.*, 2022, pp. 290–299.
- [44] Z. Yang, J. Li, Y. Ren, and P. P. C. Lee, "Tunable encrypted deduplication with attack-resilient key management," *ACM Trans. Storage*, vol. 18, no. 4, pp. 32:1–32:38, 2022.
- [45] Y. Shin, D. Koo, J. Yun, and J. Hur, "Decentralized server-aided encryption for secure deduplication in cloud storage," *IEEE Trans. Serv. Comput.*, vol. 13, no. 6, pp. 1021–1033, Nov./Dec. 2020.
- [46] J. Zhao, Z. Yang, J. Li, and P. P. C. Lee, "Encrypted data reduction: Removing redundancy from encrypted data in outsourced storage," *ACM Trans. Storage*, vol. 20, no. 4, pp. 27:1–27:30, 2024.
- [47] J. V. Bulck et al., "ForeShadow: Extracting the keys to the Intel SGX kingdom with transient out-of-order execution," in *Proc. 27th USENIX Secur. Symp.*, 2018, pp. 991–1008.
- [48] O. Oleksenko, B. Trach, R. Krahn, M. Silberstein, and C. Fetzer, "Varys: Protecting SGX enclaves from practical side-channel attacks," in *Proc. 2018 USENIX Annu. Tech. Conf.*, 2018, pp. 227–240.
- [49] J. Li et al., "Enabling secure and space-efficient metadata management in encrypted deduplication," *IEEE Trans. Comput.*, vol. 71, no. 4, pp. 959–970, Apr. 2022.
- [50] J. Park, J. Kim, Y. Kim, S. Lee, and O. Mutlu, "DeepSketch: A new machine learning-based reference search technique for post-deduplication delta compression," in *Proc. 20th USENIX Conf. File Storage Technol.*, 2022, pp. 247–264.
- [51] A. Gupta, R. Pisolkar, B. Urgaonkar, and A. Sivasubramaniam, "Leveraging value locality in optimizing NAND flash-based SSDs," in *Proc. 9th USENIX Conf. File Storage Technol.*, 2011, pp. 91–103.
- [52] J. Park, S. Lee, and J. Kim, "DAC: Dedup-assisted compression scheme for improving lifetime of NAND storage systems," in *Proc. Des. Automat. Test Europe Conf. Exhib.*, 2017, pp. 1249–1252.
- [53] OpenSSL, (n.d.). 2025. [Online]. Available: <https://www.openssl.org/>
- [54] Intel Corporation, "Intel(R) software guard extensions SDK for Linux," (n.d.). 2025. [Online]. Available: <https://01.org/intel-softwareguard-extensions>
- [55] Intel Corporation, "Intel(R) software guard extensions SSL," (n.d.). 2025. [Online]. Available: <https://github.com/intel/intel-sgx-ssl>
- [56] Traces and snapshots public archive, (n.d.). 2025. [Online]. Available: <http://tracer.filesystems.org>
- [57] Google benchmark, (n.d.). 2025. [Online]. Available: <https://github.com/google/benchmark>
- [58] M. Abadi, D. Boneh, I. Mironov, A. Raghunathan, and G. Segev, "Message-locked encryption for lock-dependent messages," in *Proc. 33rd Annu. Cryptol. Conf.*, 2013, pp. 374–391.
- [59] R. Chen, Y. Mu, G. Yang, and F. Guo, "BL-MLE: Block-level message-locked encryption for secure large file deduplication," *IEEE Trans. Inf. Forensics Secur.*, vol. 10, no. 12, pp. 2643–2652, Dec. 2015.

- [60] X. Liu, W. Sun, W. Lou, Q. Pei, and Y. Zhang, "One-tag checker: Message-locked integrity auditing on encrypted cloud deduplication storage," in *Proc. IEEE Conf. Comput. Commun.*, 2017, pp. 1–9.
- [61] Y. Zhao and S. S. M. Chow, "Updatable block-level message-locked encryption," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, 2017, pp. 449–460.
- [62] X. Liu, G. Yang, W. Susilo, J. Tonien, R. Chen, and X. Lv, "Message-locked searchable encryption: A new versatile tool for secure cloud storage," *IEEE Trans. Serv. Comput.*, vol. 15, no. 3, pp. 1664–1677, May/Jun. 2022.
- [63] G. Ha, C. Jia, Y. Chen, H. Chen, and M. Li, "A secure client-side deduplication scheme based on updatable server-aided encryption," *IEEE Trans. Cloud Comput.*, vol. 11, no. 4, pp. 3672–3684, Fourth Quarter 2023.
- [64] Y. Fu, N. Xiao, T. Chen, and J. Wang, "Fog-to-MultiCloud cooperative ehealth data management with application-aware secure deduplication," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 5, pp. 3136–3148, Sep/Oct. 2022.



Guanxiong Ha received the PhD degree in computer science and technology from Nankai University, Tianjin, China, in 2025. He is currently a lecturer with the College of Cryptology and Cyber Science, Nankai University, Tianjin, P. R. China. His research interests include encrypted deduplication, cloud data security, and applied cryptography. He has authored or coauthored several papers at premium international journals and conferences, including *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Services Computing*, *IEEE Transactions on Cloud Computing*, *Information Sciences*, and *AsiaCCS*. He has been invited as the reviewer for international journals and conferences such as *IEEE Transactions on Dependable and Secure Computing* and *IEEE ICC*.



Chunfu Jia a PhD supervisor, a professor and the head of department with the Department of Cyber Sciences, Nankai University. His main research interests include network and system security, cryptography application, and malware analysis.



Xiaowei Ge is currently working toward the master's degree with the College of Cyber Science, Nankai University, Tianjin, China. His research interest include applied cryptography.



Yuchen Chen is currently working toward the master's degree with the College of Cyber Science, Nankai University, Tianjin, China. His main research interests include applied cryptography and data security.



Xuan Shan is currently working toward the master degree with the College of Cyber Science, Nankai University, Tianjin, China. Her research interests include password security, web security, and authentication.